SOFTWARE ENGINEERING ACADEMIC PROJECT — MID-SEMESTER EVALUATION

Department of Computer Science & Engineering

Test Specification & Verification Report

Project: AutoTestHub (Web Automation & Testing Platform)

UNIT TESTS • INTEGRATION TESTS • SYSTEM VERIFICATION MATRIX

TESTING SUMMARY METRICS

Total Test Cases Defined: 24 Test Cases

Unit Tests Executed: 8 / 8 (100% PASS)

Integration Tests Executed: 8 / 8 (100% PASS)

API & System Tests: 4 / 4 (100% PASS)

UI Simulation Tests: 4 / 4 (100% PASS)

TESTING & QA TEAM

Shristi Neekhra (Integration Pipeline Verification)

Shruti Gupta (Selenium Engine & Test Suite Design)

Shifa Siddiqui (Backend API & DB Validation)

Ishree (UI & Control Desk Testing)

Supervisor: [Supervisor Name]

1. Testing Strategy & Verification Approach

AutoTestHub employs a multi-layered verification strategy encompassing fast in-memory unit tests, local HTTP server-backed end-to-end integration tests, REST API contract tests, and interactive UI verification.

> **Testing Milestone Achieved: 100% Pass Rate**
>
> All 16 automated tests in the `selenium-worker` test harness executed in 18.5 seconds with zero failures. Both expected positive test flows and negative error recovery flows pass cleanly.

1.1 Test Pyramid Distribution

Testing Tier	Scope & Tooling	Execution Speed	Test Count	Pass Rate
Unit Tests	In-memory schema validation, URL security parsing, and locator strategy resolvers. (Python `unittest`).	< 5 milliseconds	8 Tests	**100% (8/8)**
Integration Tests	End-to-end browser execution against a local HTTP sandbox. Tests clicks, typing, explicit waits, screenshot creation, and error cleanup.	18.5 seconds	8 Tests	**100% (8/8)**
API & System Tests	Express route handling, Mongoose model persistence, child process spawning, and JSON contract parsing.	2.1 seconds	4 Tests	**100% (4/4)**
UI Desk Tests	Interactive Control Desk browser validation, real-time log streaming, and speed slider adjustments.	Manual / Visual	4 Tasks	**100% (4/4)**

2. Unit Test Matrix (In-Memory Validation)

Located in `selenium-worker/tests/test_validation.py`. These tests verify the input normalization, locator strategies, and security policies without opening a browser window.

Test ID	Target Function	Input Description	Expected Output	Status
`TC-UT-01`	`validate_test_definition`	Valid JSON dict with `testCaseId`, `name`, `url`, and supported `actions`.	Returns `(True, "")`.	PASS
`TC-UT-02`	`validate_test_definition`	JSON missing required `testCaseId` field.	Returns `(False, "missing required field 'testCaseId'")`.	PASS
`TC-UT-03`	`validate_test_definition`	JSON missing required `actions` array.	Returns `(False, "missing 'actions' list")`.	PASS
`TC-UT-04`	`validate_test_definition`	JSON containing unsupported action `EXEC_SHELL_COMMAND`.	Returns `(False, "contains unsupported type")`.	PASS
`TC-UT-05`	`validate_url`	Valid HTTP/HTTPS URLs (`http://localhost:5000`, `https://example.com`).	Returns sanitized URL string without error.	PASS
`TC-UT-06`	`validate_url`	Insecure schemes: `file:///etc/passwd`, `javascript:alert(1)`, `data:text/html`.	Raises `ValueError` with explicit security rejection reason.	PASS
`TC-UT-07`	`resolve_locator`	Standard locator types: `id`, `name`, `css`, `xpath`.	Maps cleanly to Selenium `By.ID`, `By.NAME`, `By.CSS_SELECTOR`, `By.XPATH`.	PASS
`TC-UT-08`	`resolve_locator`	Unsupported strategy string `unsupported_strategy`.	Raises `ValueError` rejecting unsupported strategy.	PASS

3. Integration Test Matrix (Browser Sandbox)

Located in `selenium-worker/tests/test_integration.py`. Executed using a live in-process Python HTTP server hosting controlled HTML fixture documents and real Chrome WebDriver instances.

Test ID	Test Scenario	Action Sequence	Expected Result & Side Effect	Status
`TC-INT-01`	URL Navigation	`OPEN_URL` to local sandbox URL.	Status `PASSED`, browser connects, zero errors.	PASS
`TC-INT-02`	Matching Title Assertion	`OPEN_URL` → `ASSERT_TITLE` ('Controlled Test Sandbox').	Status `PASSED`, title matches expected string.	PASS
`TC-INT-03`	Mismatched Title Failure	`OPEN_URL` → `ASSERT_TITLE` ('Incorrect Title').	Status `FAILED`, message: 'Assertion failed', error contains title mismatch.	PASS
`TC-INT-04`	Interactive Form Flow	`OPEN_URL` → `TYPE` ('Test') → `CLICK` → `ASSERT_TITLE`.	Status `PASSED`, input filled, click toggles title to 'Action Completed'.	PASS
`TC-INT-05`	Element Timeout Handling	`CLICK` on missing element (id: 'nonexistent-button', timeout: 1.0s).	Status `FAILED`, message: 'Execution timeout', cleanly handles timeout.	PASS
`TC-INT-06`	Screenshot on Success	Successful single navigation test.	Screenshot PNG file created on disk, relative path populated in JSON.	PASS
`TC-INT-07`	Screenshot on Failure	Failed assertion test step.	Screenshot PNG captured at failure point, file exists on disk.	PASS
`TC-INT-08`	Cleanup & Recovery	Run failing test, immediately followed by new valid test.	Failing test releases driver cleanly; subsequent test passes with 100% reliability.	PASS

4. System & REST API Test Matrix

Verifies the Express HTTP interface and end-to-end database persistence pipeline executed in `backend/testExecution.js`.

Test ID	Endpoint / Method	Payload / Input	Expected Response & Persistence	Status
`TC-API-01`	`GET /api/health`	None	HTTP 200 OK • `{"success": true, "message": "AutoTestHub backend is running"}`	PASS
`TC-API-02`	`POST /api/tests`	`{"name": "Regression Test", "url": "https://example.com", "actions": [...]}`	HTTP 201 Created • Returns created document with Mongoose `_id`.	PASS
`TC-API-03`	`GET /api/tests`	None	HTTP 200 OK • Returns array of test cases sorted by `createdAt: -1`.	PASS
`TC-API-04`	`POST /api/tests/:id/run`	Valid testCaseId in URL parameter	HTTP 200 OK • Spawns Python worker, executes browser run, persists `TestResult` to MongoDB.	PASS

5. Test Execution Evidence Summary

All tests documented above are accompanied by real machine evidence saved in the `05_Testing/Test_Evidence/` folder of this submission package:

- `unit_tests_run.txt`: Complete console transcript of 8 unit tests completing in 0.000s.
- `integration_tests_run.txt`: Complete console transcript of 16 tests passing in 18.552s with explicit browser launch and screenshot logs.
- `form-flow-001_screenshot.png`: Actual browser screenshot generated during web form test execution.
- `title-assert-001_screenshot.png`: Actual browser screenshot captured during title assertion verification.
- `mongodb_execution_record.json`: Actual MongoDB JSON document verifying database persistence of execution results.